OpenAI Debugging — Worked Example

> Back to the exercise · Round 1: Basic · Round 2: Intermediate · Round 3: Real-world · Round 4: Multi-agent · Solution

One scenario, followed across all 4 rounds: Project 1's structured-extraction feature (Doc04's Build Task) and how its context gets managed. Work through the rounds in order. Each ends with **"What do you think is wrong?"** — stop and actually answer before reading on.

- Round 1: Basic
- Round 2: Intermediate
- Round 3: Real-world
- Round 4: Multi-agent

Round: Basic

Setup: a first pass at Project 1's summary feature asks nicely for JSON in the prompt, instead of using OpenAI's structured-output feature:

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system",
            "content": "Reply with JSON like {\"summary\": \"...\"}"},
        {"role": "user", "content": user_text},
    ],
)
data = json.loads(response.choices[0].message.content)
```

Symptoms: fails on effectively every run, not intermittently.

Error output:

```
Traceback (most recent call last):
  File "project_1_supportdesk_chat_and_triage/extract.py", line 33, in summarize
    data = json.loads(response.choices[0].message.content)
json.decoder.JSONDecodeError: Expecting value: line 1 column 1 (char 0)
```

`response.choices[0].message.content` actually contains:

```
Sure! Here's the summary you asked for:

{"summary": "..."}
```

Expected vs. actual:

- Expected: `response.choices[0].message.content` is exactly parseable JSON.
- Actual: the model wrapped the JSON in a friendly sentence, so `json.loads()` fails on the leading text before it ever reaches the `{`.

What do you think is wrong?

Round: Intermediate

Setup: the feature now uses real structured outputs with a Pydantic model:

```
class Summary(BaseModel):
    summary: str
    tags: list[str] = []

response = client.responses.parse(
    model="gpt-4o-mini",
    input=user_text,
    text_format=Summary,
)
```

Symptoms: this looks like a request-parsing or library-version issue — the failure happens immediately, on the API call itself, before the model even runs.

Error output:

```
openai.BadRequestError: Error code: 400 - {'error': {'message':
"Invalid schema for
response_format 'Summary': In context=('properties', 'tags'), 'default' is not
permitted.", 'type': 'invalid_request_error'}}
```

Expected vs. actual:

- Expected: `Summary` comes back as a checked object, with `tags` defaulting to an empty list when the model finds none.
- Actual: the call fails before the model ever runs, with an error about the *schema*, not about anything the model said.

What do you think is wrong?

Round: Real-world

Setup: the schema is fixed. The chatbot resends its whole conversation history on every turn (Doc04's "memory is just resent history"), with no trimming and no token counting anywhere.

Symptoms: only sometimes happens — every quick manual test (2-3 messages) works perfectly. It only shows up on genuinely long sessions, and always partway through, never at the very start.

Error output (turn 24 of a long session):

```
openai.BadRequestError: Error code: 400 - {'error': {'message':
"This model's maximum
context length is 128000 tokens. However, your messages resulted in
128412 tokens.
Please reduce the length of the messages.", 'type': 'invalid_request_error'}}
```

Expected vs. actual:

- Expected: a long-running chatbot session keeps working indefinitely, trimming its own history as needed so it never crosses the model's limit.
- Actual: it works fine for a long time, then fails abruptly on whatever turn happens to push the running total over the limit — never reproducible with a short test conversation.

What do you think is wrong?

Round: Multi-agent

Setup: the chatbot trims history correctly now. Separately, in Project 4, each of the 4 agents appends its *entire* tool-call scratchpad — every intermediate thought, every raw tool result — into the pipeline's shared state, instead of writing just its finished output there (contrary to Doc11's Core Concepts on what belongs in shared state vs. what should stay private to one agent).

Symptoms: testing the Reviewer agent alone, with a small hand-written fake state, it works fine and gives good feedback. Running the full pipeline, the Reviewer's feedback gets noticeably worse on longer tasks, and occasionally the run fails outright.

Log output:

```
[reviewer] prompt token count: 121,940
[reviewer] REJECTED: draft doesn't address the compliance concern
```

The "compliance concern" the Reviewer flagged was actually resolved in the Analysis agent's own tool-call scratchpad three steps earlier — never in Analysis agent's *finished* output, but still present in the combined prompt, buried among Research's raw search dumps and Analysis's own internal back-and-forth.

Expected vs. actual:

- Expected: each agent's prompt is built from the pipeline's clean, finished outputs — the actual conclusions each earlier agent reached.
- Actual: by the time a request reaches the Reviewer (4th in line), its prompt has ballooned with three other agents' raw internal reasoning, degrading quality and occasionally tipping the whole request over the context limit — invisible in any test that hand-builds a small, clean fake state for the Reviewer alone.

What do you think is wrong?

Full solution: Show me the solution